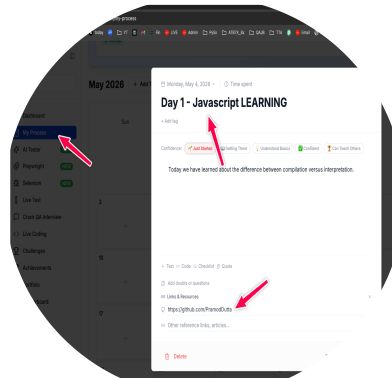


Day 2: ☐ Playwright + AI Mastery - Notes - 2x

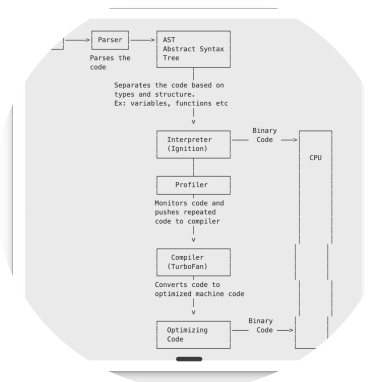


6th May 2026

☐ How JavaScript Engine Works

A JavaScript engine is a program that **reads, interprets, and executes** JavaScript code.

Here's a complete breakdown of how it works step by step.



PS. Make sure that everyone download the **Code Runner Extension**.

☐ **Step-by-Step Breakdown**

```
let x = 10;
```

The engine first breaks the raw code into small meaningful chunks called **tokens**.

1. Lexical Analysis (Tokenizing)

Example: let x = 10;

Tokens → [let] [x] [=] [10] [;]

2. Parsing (Syntax Analysis)

The tokens are then organized into a tree-like structure called an **Abstract Syntax Tree (AST)**. This represents the grammatical structure of the code.

Program

|

**** VariableDeclaration ****

/ \

**** Identifier Literal ****

(x) (10)

If there's a **syntax error**, it gets caught at this stage.

```
let a = 10;
console.log(a);
```

by breaking code into **tokens** like keywords, identifiers, operators.

Example token output:

Token	Type
let	keyword
a	identifier
=	operator
10	number literal
;	punctuation
console	identifier
.	punctuation
log	identifier
(	punctuation
a	identifier
;	punctuation

3. Create AST (Abstract Syntax Tree)

The tokens are converted into a structured tree representing the logic.

Simplified AST example:

```
Program
├── VariableDeclaration (let)
│   ├── Identifier: a
│   └── Literal: 10
└── ExpressionStatement
    └── CallExpression: console.log
        └── Identifier: a
```

astexplorer.net/ (<https://astexplorer.net/>).

ast-explorer.dev/ (<https://ast-explorer.dev/>).

4. Interpreter (Ignition) Converts AST to Bytecode

Ignition creates lightweight bytecode from AST and starts executing immediately.

Bytecode might look like:

```
LdaSmi [10]      // Load immediate small integer 10
Star <a>          // Store into variable a
LdaGlobal console
LdaNamedProperty log
Lda <a>           // Load value of a
CallProperty1    // Call log with 1 argument
```

. Execution (Runtime with Call Stack,

The **Call Stack** executes bytecode line-by-line:

Step	Action
1	Allocate memory for <code>a</code>
2	Store value <code>10</code> in memory
3	Evaluate <code>console.log(a)</code>
4	Fetch value <code>10</code> from memory
5	Send value to console output
6	Prints <code>10</code> in terminal/browser console

Output:

10

6. Profiler + JIT Optimization

Since this code is very small and not repeated, the **Profiler sees no benefit**, so **TurboFan (JIT Compiler) does NOT optimize**.

If we had a loop with thousands of executions, V8 would compile that part into machine code.

Profile. - your responsibility is to check if the code needs optimization or not

Hot Code -> Code which needs optimization

TurboFan -> Compiler convert or optimize the code of the Hot code. -> ByteCode.

JIT -> Just in time

7. Garbage Collector

Automatically frees memory by removing unused or unreachable objects. Prevents memory leaks and keeps performance efficient.

8. Machine Code Execution

The final optimized code is executed directly by the **CPU**. This gives the best performance at runtime.

How to See the Bytecode?

```
node --print-bytecode chapter_01_Basics/01_Basics.js
```

How to push the code daily to your own Github Repo?

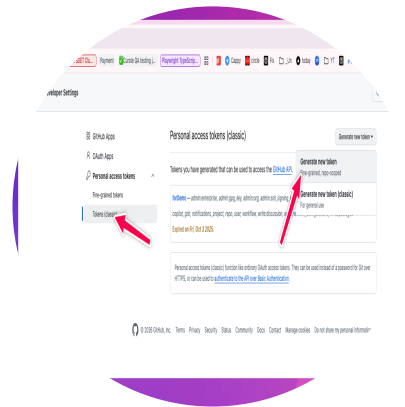
- Step Make sure do have an account at Github? If you don't have an account, try going to github.com/signup (<https://github.com/signup>).
- github.com/new (<https://github.com/new>)

Rather than the password, sometimes it will ask you for a token.

github.com/settings/tokens (<https://github.com/settings/tokens>)

pramoddutta

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



1. Give it a try by yourself
2. if it is not working, we will have a session for it, everyone. How many of you are with me
3. OpenCode Issues / Github Coplit
4. Not Able to Run the program
5. Not able to Github Push. github.com/PramodDutta/LearnPlaywrightBatch2x/tree/main/chapter_01_Basics
(https://github.com/PramodDutta/LearnPlaywrightBatch2x/tree/main/chapter_01_Basics).

Today - 8PM-9PM IST - Link (Doubt Class) - 1hour - Link I will share with you.

